```
class function_wrapper
{
    struct impl_base {
        virtual void call()=0;
        virtual ~impl_base() {}
    };
    std::unique_ptr<impl_base> impl;
    template<typename F>
    struct impl_type: impl_base
    {
        F f;
        impl_type(F&& f_): f(std::move(f_)) {}
        void call() { f(); }
    };
public:
    template<typename F>
    function_wrapper(F&& f):
        impl(new impl_type<F>(std::move(f)))


    {}

    void operator()() { impl->call(); }

    function_wrapper() = default;

    function_wrapper(function_wrapper&& other):
        impl(std::move(other.impl))
    {}
    function_wrapper& operator=(function_wrapper&& other)
    {
        impl=std::move(other.impl);
        return *this;
    }

    function_wrapper(const function_wrapper&)=delete;
    function_wrapper(function_wrapper&)=delete;
    function_wrapper& operator=(const function_wrapper&)=delete;
};
```

First, the modified `submit()` function ❶ returns a `std::future<>` to hold the return value of the task and allow the caller to wait for the task to complete. This requires that you know the return type of the supplied function `f`, which is where `std::result_of<>` comes in: `std::result_of<FunctionType()>::type` is the type of the result of invoking an instance of type `FunctionType` (such as `f`) with no arguments. You use the same `std::result_of<>` expression for the `result_type typedef` ❷ inside the function.